

Session 01 - Setup and Foundations

payments-platform · Arihant Rakhecha

15 September 2026

- [How to read this](#)
- [Part 1 – What we are building, in plain English](#)
 - [The one-sentence version](#)
 - [Why that is hard](#)
 - [The two halves of the system](#)
- [Part 2 – The vocabulary you need first](#)
 - [Terminal, shell, and command](#)
 - [PATH – and why `claude: command not found` happened](#)
 - [Homebrew – the package manager](#)
 - [Files, folders, and the two dot-files that matter](#)
- [Part 3 – Containers, and why we did not just install Postgres](#)
 - [The problem containers solve](#)
 - [What a container actually is](#)
 - [alpine – why `postgres:16-alpine` and not `postgres:16`](#)
 - [Docker Compose and the `docker-compose.yml` file](#)
 - [Ports – and the `9092 / 29092` puzzle](#)
 - [Volumes – where data survives](#)
 - [KRaft – why no Zookeeper](#)
 - [The RAM problem and what we did about it](#)
- [Part 4 – Every problem we hit, and the reasoning that solved it](#)
 - [Problem 1 – `zsh: command not found: claude`](#)
 - [Problem 2 – `error: unknown option '--global'`](#)
 - [Problem 3 – the download was called `files.zip`](#)
 - [Problem 4 – `zsh: permission denied: ./scripts/init-repo.sh`](#)
 - [Problem 5 – `Failed building wheel for pydantic-core`](#)
 - [What a virtual environment is, since it came up](#)
 - [Problem 6 – `make: docker: No such file or directory`](#)
 - [Problem 7 – `\[spinner\] Formula xz \(5.8.4\) appeared frozen`](#)
 - [Problem 8 – GitHub said “Bengaluru” when you are in Hyderabad](#)
 - [The overall pattern](#)
- [Part 5 – The design ideas in the code](#)
 - [Double-entry bookkeeping](#)
 - [Money as integers](#)
 - [Idempotency](#)
 - [Kafka, and why it is not a queue](#)
 - [Partitions and keys](#)
 - [Consumer groups](#)
 - [The transactional outbox – the centrepiece](#)
 - [Why the API does not know Kafka exists](#)
 - [Row locking and the deadlock that was designed out](#)
 - [`FOR UPDATE SKIP LOCKED`](#)
 - [The state machine](#)
- [Part 6 – Every command we ran, and why that one](#)
 - [Installation](#)
 - [Git and GitHub](#)
 - [Running the project](#)
 - [Diagnostic commands worth memorising](#)
 - [`&` and `kill`](#)
- [Part 7 – Where you actually are](#)
 - [Still to verify](#)
 - [Next](#)
- [Glossary](#)

How to read this

This covers only what was actually done in session 1. Nothing aspirational.

Read it in order the first time. After that, the Glossary at the end is the part you will come back to.

Whenever a decision was made, there is a **“Why not the alternative”** note. That is the part interviewers ask about. Knowing *what* you built is table stakes; knowing *what you rejected and why* is the thing that separates people who built a project from people who followed a tutorial.

Part 1 - What we are building, in plain English

The one-sentence version

A backend system that moves money between accounts without ever losing or duplicating a rupee, even when parts of it crash.

Why that is hard

Imagine you pay a merchant Rs 2,500. Three things must happen:

1. Rs 2,500 leaves your balance.
2. Rs 2,500 arrives in the merchant's balance.
3. Other parts of the system get told about it (send a receipt, update a dashboard, run fraud checks).

If the computer dies after step 1 but before step 2, the money has vanished. Not “temporarily unavailable” – genuinely gone, with no record of where. That is the failure mode the entire design exists to prevent.

Almost every technique in this project is a defence against some version of “what if it crashes *right here*”.

The two halves of the system

The ledger is the record of truth. It is an append-only list of money movements. Append-only means you never edit or delete a row – if you made a mistake, you add a new row that cancels the old one. Like a bank passbook: you do not erase a wrong entry, you add a correcting one. This matters because an auditor needs to see not just what you believe now, but what you believed last Tuesday and when you changed your mind.

The event stream is how the rest of the system finds out. When a payment happens, an “event” is published – a small message saying “payment X was authorized”. Other programs subscribe to that stream and react.

Later, a trading module (buy/sell orders for stocks) gets built on the same ledger, because a stock position is just another kind of account.

Part 2 - The vocabulary you need first

This section exists because the rest of the document is unreadable without it. None of these are difficult ideas. They are just words nobody defines.

Terminal, shell, and command

The **terminal** is the black window. It is only a window – a display.

The **shell** is the program running *inside* that window that reads what you type and does it. Yours is `zsh`. That is why errors say `zsh: command not found` – the shell is the thing complaining, not the terminal.

A **command** is the first word you type. Everything after it is arguments.

```
mkdir -p ~/Developer
```

- `mkdir` – the command (“make directory”)
- `-p` – a **flag**, a switch that modifies behaviour
- `~/Developer` – the argument, what to operate on

`~` means your home folder, `/Users/arihantrakhecha`. It is shorthand; typing `~/Developer` and `/Users/arihantrakhecha/Developer` are identical.

What `-p` actually did: without it, `mkdir` fails if the parent folder does not exist, and also fails if the folder *already* exists. With `-p` it creates missing parents and stays silent if the target is already there. That second property is why it is used in scripts – you can run the script twice without it erroring. This idea (safe to run twice) is called **idempotency** and it comes back as a major theme in Part 5.

PATH - and why `claude: command not found` happened

When you type `claude`, the shell does **not** search your whole computer. That would be far too slow. Instead it checks a specific list of folders, in order, and uses the first match. That list is the `PATH` variable.

See yours:

```
echo $PATH | tr ':' '\n'
```

`echo` prints something. `$PATH` is the variable’s value. `|` is a **pipe** – it feeds the output of the left command into the right one as input. `tr ':' '\n'` translates every colon into a newline, so a single unreadable line becomes one folder per line.

Pipes are one of the most useful ideas in the terminal. Each command does one small thing, and you chain them.

The bug we hit: `claude` was installed at `/opt/homebrew/bin/claude`, and `/opt/homebrew/bin` was in your `PATH`. But the shell keeps a **cache** of where commands live, built when the shell started. Your shell had started *before* the install, so its cache had no entry for `claude`, and it reported “not found” without re-checking the folder.

```
hash -r
```

`hash -r` clears that cache. Instantly fixed. Opening a brand new terminal tab would also have worked, because a new shell builds a fresh cache.

Why not just reinstall? That was the instinct, and it would have wasted several minutes downloading something already present. The diagnostic `which claude` was the right first move: it asks “where would the shell find this?” and answered `/opt/homebrew/bin/claude`. That single line proved the file existed, which eliminated “not installed” and pointed straight at `PATH` or caching. **Diagnose before you fix.** Reinstalling is the equivalent of restarting your laptop – sometimes it works, but you learn nothing and you often have not fixed the real thing.

Homebrew - the package manager

On Windows you download `.exe` files from websites. On macOS you *can* do that, but it means hunting for download links, no record of what you installed, and no easy way to update.

A **package manager** is a program that installs other programs for you from a central catalogue. Homebrew (`brew`) is the standard one for macOS.

```
brew install gh
```

Two kinds of thing Homebrew installs:

- **Formula** - command-line tools. `brew install gh`
- **Cask** - normal Mac applications with a window and an icon. `brew install --cask docker-desktop`

That distinction caused a real problem, covered in Part 4.

Files, folders, and the two dot-files that matter

A file starting with `.` is hidden by default. `ls` will not show it; `ls -la` will. The `-a` means “all”.

Two in this project:

- `.env` - holds configuration and secrets. Never goes on GitHub.
- `.env.example` - same keys, fake values. *Does* go on GitHub, so someone cloning your repo knows what settings exist without you leaking yours.

This pair is a standard convention and worth internalising. It solves a real tension: you need to document what configuration exists, but you must not publish the actual values.

`.gitignore` lists things Git should pretend it cannot see. Ours contains `.env`, which is why `init-repo.sh` paused and made you check the file list. Committed secrets are the single most common embarrassing mistake in student repositories, and once pushed they are in the history forever even if you delete the file later.

Part 3 - Containers, and why we did not just install Postgres

This is the concept most worth understanding properly, because it explains about half of what we typed.

The problem containers solve

The project needs four separate server programs: PostgreSQL (the database), Redis (fast temporary storage), Kafka (the event stream), and later monitoring tools.

Installing those directly on your Mac would mean:

- Four separate installations, each with its own config files in its own place
- Version conflicts – if another project needs Postgres 14 and this needs 16, you are stuck
- Background processes running and eating RAM even when you are doing DSA
- No clean way to reset when something breaks
- “Works on my machine” – your setup drifts from anyone else’s, and bugs appear that nobody can reproduce

What a container actually is

A **container** is a program plus everything it needs to run – its libraries, its config, its filesystem – packaged into one isolated bundle. From inside the container, the program believes it has a whole Linux machine to itself. In reality it is sharing your Mac’s kernel with the other containers.

The crucial bit: **it is isolated**. The Postgres in your container cannot see your Mac’s files and does not care what else is installed. Delete the container and every trace goes with it.

Image vs container (this trips everyone up):

- An **image** is the read-only template. `postgres:16-alpine` is an image. Like a class in programming, or an ISO file.
- A **container** is a running instance of an image. Like an object.

One image, many containers. That is why `make up` printed `Image postgres:16-alpine Pulled` (downloaded the template) and then `Container pp-postgres Started` (made a running copy).

Why not a virtual machine? You already run one – the Ubuntu VM in UTM for your Linux coursework. A VM emulates an entire computer including its own kernel, so it needs gigabytes of RAM and a minute to boot. Containers share your Mac’s kernel, so they start in about a second and use a fraction of the memory. On an 8 GB machine that difference is not a nicety, it is the difference between the project running and not running. The trade-off is that containers can only run Linux programs (which is all we need) whereas a VM can run any OS.

`alpine` – **why** `postgres:16-alpine` **and not** `postgres:16`

The part after the colon is the **tag**, identifying which version of the image.

`alpine` means it is built on Alpine Linux, a deliberately minimal Linux distribution. The regular `postgres:16` image is roughly 400 MB because it includes a full Debian system. The `alpine` variant is around 80 MB because it strips everything Postgres does not strictly need.

Chosen for three reasons: less to download, less disk used, and a smaller **attack surface** – fewer programs installed means fewer things that can have security holes. In production that last reason is the important one.

The trade-off: if you ever need to debug *inside* the container, the minimal image lacks tools you might reach for. Acceptable here.

Docker Compose and the `docker-compose.yml` file

Starting four containers by hand means four long commands, each specifying ports, passwords, environment variables and network settings. Error-prone and impossible to remember.

Docker Compose reads a file describing all of them and starts them together. `docker-compose.yml` is that file. YAML is a

text format where indentation defines structure – which means **indentation is not cosmetic, it is syntax**. Two spaces wrong and the file means something different.

```
docker compose up -d
```

- `up` – create and start everything in the file
- `-d` – **detached**. Run in the background and give the terminal back.

Without `-d`, all four programs’ logs would pour into your terminal and you could not type anything until you pressed Ctrl+C, which would also stop them.

Ports – and the 9092 / 29092 puzzle

A **port** is a numbered door on a machine. One machine has one address but 65,535 ports, so many programs can each listen on their own without clashing. Postgres conventionally uses 5432, Redis 6379, Kafka 9092.

In `docker-compose.yml`:

```
ports:
  - "5432:5432"
```

This is `host:container` – “connect port 5432 on my Mac to port 5432 inside the container”. Without this line the container would run perfectly but be unreachable from your Mac, because containers are isolated by default.

Kafka needed something stranger:

```
ports:
  - "29092:29092"
```

Kafka has a genuinely awkward property. When a client connects, Kafka does not just serve the request – it **replies with the address clients should use for future connections**. This is because a real Kafka cluster has many brokers and has to tell clients which one owns which data.

So the address Kafka advertises must be correct *from the caller’s perspective*. But there are two different perspectives:

- Another container calls it `kafka` (the service name) on port 9092
- Your Mac calls it `localhost` on port 29092

One advertised address cannot satisfy both. If Kafka advertised `kafka:9092`, your Python code on the Mac would connect, get told “talk to `kafka:9092`”, and fail – your Mac has no idea what `kafka` means. If it advertised `localhost:29092`, containers would fail for the mirror-image reason.

The fix is two **listeners**, `PLAINTEXT` for container-to-container and `PLAINTEXT_HOST` for your Mac, each advertising the address appropriate to its audience. This is the single most common Kafka setup problem and the reason `KAFKA_ADVERTISED_LISTENERS` appears in every Kafka tutorial’s troubleshooting section.

Volumes – where data survives

```
volumes:
  - pgdata:/var/lib/postgresql/data
```

Container filesystems are **ephemeral**: delete the container and the data goes with it. A **volume** is storage managed by Docker that lives outside the container, so the data survives `docker compose down` and a restart.

Consequence worth knowing: `make down` runs `docker compose down -v`, and the `-v` deletes volumes too. That is deliberate – during development you often want a genuinely clean slate. But it means `make down` wipes your database. Not a problem now; would be a problem if you had data you cared about.

KRaft – why no Zookeeper

Every Kafka tutorial older than about 2024 starts a second service called Zookeeper alongside Kafka. Zookeeper kept

track of cluster coordination: which broker is in charge, where data lives.

Kafka now does this itself, in a mode called **KRaft**. Our compose file has no Zookeeper, which is why the `kafka` service has confusing settings like `KAFKA_PROCESS_ROLES: broker,controller` – it is playing both parts.

Why not Zookeeper? It is being removed from Kafka entirely. Using it would mean learning a component that is on its way out, running a second JVM process on an 8 GB laptop, and producing a project that looks dated to anyone who knows current Kafka. The only reason to pick it would be following an old tutorial, which is not a reason.

The RAM problem and what we did about it

Kafka runs on the JVM (Java Virtual Machine) and by default reserves a **1 GB heap** – memory set aside for itself – whether it needs it or not. At our scale it needs nowhere near that.

Your machine has 8 GB total, shared between macOS, Docker Desktop, Postgres, Redis, Kafka, your Python processes, a browser, and your DSA work. So:

```
KAFKA_HEAP_OPTS: "-Xmx512m -Xms512m"
```

Halves Kafka's appetite with no practical downside here.

We also moved `kafka-ui` (a web dashboard for inspecting Kafka) behind a **profile**, meaning it only starts if you explicitly ask via `make ui`. It is a *second* JVM, and you rarely need it – you can read topics from inside the broker container instead.

That is why `make up` started three containers, not four. Not a bug.

Part 4 - Every problem we hit, and the reasoning that solved it

This is the most useful section for you. Solutions are less valuable than the *diagnostic reasoning*, so each one shows the thinking.

Problem 1 - `zsh: command not found: claude`

Cause: the shell's command cache was built before the install.

Diagnosis: `which claude` printed a path, proving the file existed. That one line eliminated “not installed”.

Fix: `hash -r`.

The transferable lesson: “command not found” has three distinct causes and they need different fixes – not installed, installed but not in PATH, or in PATH but cached. `which` distinguishes them in one step.

Problem 2 - `error: unknown option '--global'`

Cause: `claude config set --global` existed in older versions and was removed.

Fix: write `~/.claude/settings.json` directly.

Lesson: command-line tools change. An error naming an option you were told to use usually means the instruction is stale, not that you typed it wrong. The underlying config file is almost always more stable than the convenience command that writes it.

Problem 3 - the download was called `files.zip`

Cause: downloading several files at once bundles them, and the browser named the bundle generically.

Fix:

```
ls -lt ~/Downloads | head -5
```

`ls` lists, `-l` gives the long format (size, date, permissions), `-t` sorts by modification time newest-first, and `| head -5` keeps only the first five lines.

Lesson: when a file “is not there”, list the folder sorted by time rather than guessing names. `-t` is what makes this work – the thing you just downloaded is at the top by definition.

A subtlety: `~/Downloads/"files 2"` needed quotes because the folder name contains a space. Without quotes the shell reads `files` and `2` as two separate arguments. This is why programmers avoid spaces in filenames.

Problem 4 - `zsh: permission denied: ./scripts/init-repo.sh`

Cause: every file carries **permission bits**, including whether it may be executed. Zip archives store these inconsistently and macOS's Archive Utility often drops them. The script arrived readable but not executable.

Fix:

```
chmod +x scripts/init-repo.sh scripts/demo.sh
```

`chmod` = change mode, `+x` = add execute permission.

The alternative, and why either works:

```
bash scripts/init-repo.sh
```

This succeeds because you are running `bash` – which *is* executable – and handing it the script as a plain text argument.

The script's own permission bits are never consulted.

We used `chmod` because `make demo` later calls `scripts/demo.sh` internally and would hit the identical wall. Fixing the files is better than working around them each time.

It also had a good side effect. Your commit shows `create mode 100755 scripts/demo.sh`. The `755` is the permission in octal, and the leading `7` means executable. Git recorded it, so anyone cloning your repo gets working scripts. You fixed it for every future user, not just yourself.

Problem 5 - Failed building wheel for pydantic-core

The most instructive failure of the session.

The error, decoded. Buried in the output: `pydantic_core._pydantic_core.cpython-314-darwin.so`. That `314` is Python **3.14**. Your default `python3` was 3.14, and the project targets 3.12.

Why that broke things. Most Python packages are pure Python and work anywhere. But `pydantic-core` is written in **Rust** for speed. It ships **wheels** – precompiled binaries, one per Python version and per platform. There is no wheel for Python 3.14 at the pinned version, so `pip` fell back to compiling from Rust source, which needs a Rust toolchain you do not have.

The fix:

```
deactivate
rm -rf .venv
brew install python@3.12
/opt/homebrew/bin/python3.12 -m venv .venv
source .venv/bin/activate
python --version
```

Why not upgrade pydantic to a 3.14-compatible version? This was the tempting option and it was the wrong one. Upgrading pydantic forces compatible versions of SQLAlchemy, asyncpg, aiokafka and their dependencies – some of which have no Python 3.14 wheels at all yet. You would spend the evening resolving a dependency graph and end up on versions whose behaviour you have not read about. Python 3.14 is very new; the ecosystem lags. Moving the interpreter down is one command; dragging the whole ecosystem up is open-ended. **Change the one thing, not the twenty.**

Why we deleted `.venv` instead of reusing it. A virtual environment is hard-wired to one interpreter at creation time. Its `bin/python` is a link to Python 3.14, and there is no supported way to re-point it. Deleting and recreating is the only clean route. Cheap anyway – everything reinstalls from wheels in seconds.

How we knew the fix worked. Look at the successful output: `pydantic_core-2.27.2-cp312-cp312-macosx_11_0_arm64.whl`. Three things confirmed at a glance:

- `cp312` – built for Python 3.12, matching our interpreter
- `arm64` – built for Apple Silicon, matching your M2
- `.whl` and “Downloading”, not “Building wheel for” – a prebuilt binary, no compilation

Lesson: read error output bottom-to-top for the summary, then hunt upward for the first line mentioning a version or a path. `cpython-314` was the whole answer and it was sitting in plain sight.

What a virtual environment is, since it came up

```
python3.12 -m venv .venv
source .venv/bin/activate
```

By default `pip install` puts packages in one shared system-wide location. With several projects that breaks quickly: project A needs SQLAlchemy 1.4, project B needs 2.0, and only one can win.

A **virtual environment** is a private folder holding its own Python and its own packages. `activate` adjusts your PATH so

`python` and `pip` point inside it. Your prompt gains `(.venv)` as a reminder.

Why `source` and not just running it? Running a script normally starts a new shell, which changes *its* environment and then exits, discarding the change. `source` executes the commands in your *current* shell, so the PATH change persists. This is the same reason `cd` inside a script does not change your terminal's folder.

`deactivate` reverses it.

Problem 6 - `make: docker: No such file or directory`

Cause: Homebrew renamed the cask. `brew install --cask docker` now refers to the CLI-only formula, not Docker Desktop.

Fix: `brew install --cask docker-desktop`.

Lesson: “command not found” *after* an apparently successful install often means you installed something adjacent to what you wanted. Package names drift.

Problem 7 - `[spinner] Formula xz (5.8.4) appeared frozen`

Diagnosis: everything above it read `Bottle ... Downloaded`. A **bottle** is a prebuilt binary from Homebrew's CDN – fast. But the `xz` line read `API Source xz.rb`, a different and slower path. Different mechanism, so slowness was expected.

Fix: waited. It finished.

Lesson: before killing a process, look for evidence it is doing something different rather than nothing. Ctrl+C on a half-finished install can leave a broken state that is worse than waiting.

Problem 8 - GitHub said “Bengaluru” when you are in Hyderabad

Not a problem, but worth understanding.

IP geolocation locates the **network exit point**, not you. Indian ISPs and campus networks aggregate traffic regionally and route it out through a few data centres. Bengaluru is a major peering hub, so a great deal of South Indian traffic surfaces there.

What to actually check on that screen: not the city, but “did I just run `gh auth login`?” You had. The city is weak evidence on Indian IPs.

The overall pattern

Every one of these followed the same shape:

1. **Read the actual error**, not the general shape of it. `cpython-314` and `API Source xz.rb` were both the entire answer, in plain sight.
2. **Form a hypothesis** about the cause.
3. **Run one cheap command to test it** – `which`, `ls -lt`, `python --version`.
4. **Fix the specific cause.**

The failure mode to avoid is jumping from symptom to fix. Reinstalling, restarting, or pasting a Stack Overflow command sometimes works, and teaches you nothing about why.

Part 5 - The design ideas in the code

You have not read the code yet. This is so the words make sense when you do.

Double-entry bookkeeping

Every movement is recorded **twice**: once as money leaving, once as money arriving. The two must sum to exactly zero.

Paying Rs 2,500:

```
-250000    your account      (in paise)
+250000    clearing account
-----
          0    must always be zero
```

Why bother, when one “balance” column would do? Because a single column can be wrong silently. If a bug adds 500 to one account and nothing anywhere else, nothing detects it – the number just looks like a number. With double-entry, that same bug produces a transaction summing to 500 instead of 0, which is arithmetically impossible and therefore checkable. The system can *prove* it is consistent rather than hope.

This is a 700-year-old technique from Italian merchant accounting. It survives because the zero-sum property makes errors detectable rather than invisible.

Money as integers

Amounts are stored as whole numbers of **paise**, never rupees with decimals. Rs 2,500 is stored as `250000`.

Why: computers store decimals in binary, and most decimal fractions have no exact binary representation. `0.1 + 0.2` genuinely evaluates to `0.30000000000000004`. Over millions of rows those errors accumulate and the ledger stops summing to zero – destroying the one property that made it checkable.

Integers are exact. There is no rounding to get wrong.

Why not NUMERIC ? Postgres has a `NUMERIC` type that is exact for decimals, and it is a defensible choice. Integers were picked because they are exact *and* cheap to add, index and compare, and because they force you to decide explicitly where a remainder goes when splitting a fee. With `NUMERIC` that decision hides inside a rounding mode nobody reads. Making it visible is a feature.

Idempotency

An operation is **idempotent** if doing it twice has the same effect as doing it once.

This is not academic. Your phone sends “pay Rs 2,500”, the network drops the reply, and your app retries. The server has now received the same instruction twice. **The client genuinely cannot distinguish a lost request from a lost reply**, so it has no safe choice but to retry – which means the *server* must be the thing that makes retrying safe.

The mechanism: the client sends a unique **idempotency key**. The server has a database rule (a `UNIQUE` index) making it impossible to store two payments with the same key. The second attempt is rejected by the database and returns the first payment’s result.

Why a database constraint and not `if not exists: insert` ? Because that has a race window. Two retries arrive at the same instant, both check, both see nothing, both insert. You have double-charged someone. Only the database can make the check and the insert genuinely atomic. **When correctness depends on two things not happening at once, application code cannot be the guard.**

You saw this proved: `make demo` sends the same key twice and gets the same payment back, then captures twice and the second returns `409 Conflict`.

Kafka, and why it is not a queue

A **queue** (RabbitMQ, SQS) delivers a message, the consumer acknowledges, the message is **deleted**. Good for handing out work.

Kafka is a log. Messages are appended and *retained*. Each reader tracks its own position, called an **offset**. Multiple independent readers can each read everything at their own pace.

That difference buys three things a queue structurally cannot:

1. **Many independent consumers.** A settlement service, a dashboard builder and a fraud checker each need every event. In a queue, one consumer's acknowledgement destroys the message for the others.
2. **Replay.** If your dashboard's data gets corrupted, you rebuild it by re-reading from offset 0. A queue has no history left to re-read.
3. **Per-entity ordering with parallelism.** Explained next.

Why not RabbitMQ? The original plan was RabbitMQ. It is simpler to operate and has features Kafka lacks (per-message delay, TTL). But this system wants the event log to *be* the source of truth, and a queue that deletes messages cannot serve that role. The cost accepted: Kafka is heavier, and because it has no native delay primitive, retries need an explicit extra topic – which is milestone 3.

Partitions and keys

A **topic** (our `payments.payment.v1`) is split into **partitions**, and partitions are what allow parallelism – different partitions can be processed simultaneously.

Kafka guarantees ordering **within a partition only**. Which partition a message lands in is decided by its **key**.

We key on the payment ID. So every event for payment X lands in the same partition and arrives in order, while different payments spread across partitions and process in parallel. Ordering where needed, parallelism everywhere else.

The counterexample that shows why this matters. With no key, messages are spread round-robin.

`payment.authorized` and `payment.captured` for the *same* payment land in different partitions and can arrive in either order. Your code receives “captured” for a payment it has never seen authorized, and correctly rejects a legitimate transaction. This is the most common Kafka bug there is.

Consumer groups

Consumers sharing a `group_id` **split** the partitions between them – add more copies of the program and throughput rises. Consumers with *different* group IDs each get the **full** stream independently.

That is the mechanism behind “add a fraud checker later without touching anything”: new group ID, reads everything from the beginning, existing consumers unaffected and unaware.

The transactional outbox – the centrepiece

The problem. A payment must (a) change rows in Postgres and (b) publish an event to Kafka. These are two separate systems with no shared transaction, so they cannot both be guaranteed. Only bad orderings exist:

- **Save, then publish.** Crash in the gap: the payment exists, no event ever fires. Nothing downstream knows. Silent, permanent, undetectable later – the worst possible failure shape.
- **Publish, then save.** The save fails and rolls back, but you have already announced a payment that never happened. Downstream acts on a fiction.

The solution. Do not publish from the API at all. Write the event as a **row in the database**, in the same transaction as the payment. Either all of it lands or none of it does – one database, one transaction, guaranteed.

A separate program, the **relay**, then reads unpublished rows and pushes them to Kafka, marking them published as it goes.

Why this is safe: the relay can crash freely. On restart it finds rows still marked unpublished and tries again.

The cost, accepted deliberately. The relay might crash *after* publishing but *before* marking the row, so on restart it publishes the same event twice. This is **at-least-once** delivery. It is not a flaw we failed to fix; it is a trade we chose, because the alternative (losing events) is unacceptable while duplicates are merely inconvenient.

Duplicates are then handled on the consumer side: every event has a unique ID, and the consumer records processed IDs in a table *in the same transaction* as its actual work. A duplicate hits the primary key and is skipped.

At-least-once delivery plus idempotent consumers gives you **effectively-once**, which is what “exactly-once” means in practice. Worth being able to say out loud.

Why not Debezium? Debezium reads Postgres’s write-ahead log directly and publishes changes with no polling and no relay to run. It is arguably the better production answer. Rejected here because running Debezium plus Kafka Connect is two more services on an 8 GB laptop, and it hides the mechanism this project exists to demonstrate. Recorded in ADR 0002 as worth revisiting.

Why the API does not know Kafka exists

A consequence worth noticing: the API only writes to Postgres. If Kafka is completely down, payments still succeed – the events queue up in the outbox and flush when Kafka returns.

This is also why the health check tests Postgres but deliberately **not** Kafka. If it checked Kafka, a broker outage would fail the health check, the load balancer would remove every instance, and the whole API would go down – throwing away the exact resilience the outbox bought you.

Row locking and the deadlock that was designed out

When two payments touch the same account simultaneously, both could read the old balance and one update overwrites the other. The fix is a **lock**: the first transaction claims the row, the second waits.

But locks create a new danger. Transaction A locks account X then wants Y. Transaction B locks Y then wants X. Neither can proceed. That is a **deadlock**. Postgres detects it and kills one at random, so under load you get unexplained failures that do not reproduce when you try them one at a time.

The fix is one line in `ledger.py`:

```
account_ids = sorted(merged.keys(), key=str)
```

If *everyone* locks in the same agreed order, the circular wait cannot form. `sorted()` looks cosmetic and is load-bearing.

`FOR UPDATE SKIP LOCKED`

The relay uses this when claiming outbox rows. `FOR UPDATE` locks them. `SKIP LOCKED` means “if a row is already locked by someone else, ignore it and take the next one”.

That is what lets you run several relays at once – each grabs a different batch. Without `SKIP LOCKED`, a second relay would simply block on the rows the first holds: pure contention, zero extra throughput.

The state machine

A payment moves through states: `initiated` -> `authorized` -> `captured`. Refunds and voids branch off. Some states are **terminal** – nothing follows them.

The legal transitions live in one dictionary in `state_machine.py`, not scattered across `if` statements. **Why:** when the rules are spread through ten conditionals, someone eventually writes an eleventh that allows capturing a voided payment, and nobody notices.

One subtle choice: `captured > captured` is treated as **illegal**, not a harmless no-op. A second capture is either a retry (which idempotency should have caught earlier) or a bug. Silently succeeding hides both.

Why authorize and capture are separate at all. Between them, the money sits in a third **clearing** account – in neither party’s spendable balance. That gap is real: a card network responds in seconds but a warehouse ships in days. A

single payer-to-payee transfer would be simpler and would not model reality.

Part 6 - Every command we ran, and why that one

Installation

Command	What it does	Why this and not something else
<code>brew install --cask claude-code</code>	Installs Claude Code	Homebrew gives one uninstall path and a record. Downside: no auto-update, so <code>brew upgrade</code> occasionally. The <code>curl bash</code> installer auto-updates but leaves nothing Homebrew knows about.
<code>brew install gh</code>	GitHub's official CLI	Creates repos and handles auth from the terminal. Without it you click through github.com and paste a remote URL by hand.
<code>brew install --cask docker-desktop</code>	Docker Desktop	<code>--cask docker</code> now means the CLI-only formula. Name drift caused Problem 6.
<code>brew install python@3.12</code>	Python 3.12	Fixed the wheel failure. Alternative: <code>uv</code> , which downloads standalone Pythons and is far faster - worth knowing for next time.

Git and GitHub

Command	What it does	Why
<code>git config --global user.name</code>	Sets commit author name	<code>--global</code> applies to every repo, so you set it once. Display name, not username.
<code>git config --global user.email</code>	Sets commit author email	Must match your GitHub email or commits are not attributed to you.
<code>gh auth login</code>	Authenticates	Chose HTTPS over SSH: SSH needs a keypair generated and uploaded first. Better long-term; three extra steps tonight.
<code>git init -b main</code>	Creates the repository	<code>-b main</code> names the first branch explicitly. Older Git defaults to <code>master</code> ; being explicit avoids depending on the version.
<code>git add .</code>	Stages everything	<code>.</code> means the current folder recursively. Safe only because <code>.gitignore</code> excludes <code>.env</code> - which is exactly why the script paused for you to check.
<code>git commit -m "..."</code>	Records a snapshot	<code>-m</code> supplies the message inline. Without it, Git opens an editor (often <code>vim</code>), which is where people get stuck.
<code>gh repo create --source=. --push</code>	Creates on GitHub and pushes	One command instead of: create in browser, copy URL, <code>git remote add</code> , <code>git push -u</code> .

Running the project

Command	What it does	Why
<code>python3.12 -m venv .venv</code>	Creates the isolated environment	<code>-m</code> runs a module. Guarantees the <code>venv</code> belongs to <i>that</i> interpreter.
<code>source .venv/bin/activate</code>	Enters it	<code>source</code> runs in the <i>current</i> shell so the <code>PATH</code> change survives.
<code>pip install -r requirements.txt</code>	Installs pinned dependencies	<code>-r</code> reads the file. Pinned exact versions so the install is reproducible - an unpinned install would eventually break on a future

		release.
<code>cp .env.example .env</code>	Creates your local config	<code>cp</code> copies. Gives you a real config from the committed template.
<code>make up</code>	Starts the containers	<code>make</code> runs recipes from the <code>Makefile</code> . Why not type <code>docker compose up -d</code> ? Because the real commands get long, and <code>make topics</code> is four lines you would otherwise have to remember exactly. Self-documenting.
<code>make migrate</code>	Creates the database tables	Runs Alembic. Why not hand-written CREATE TABLE ? Migrations are versioned and ordered, so the schema can be rebuilt from scratch identically, and changes are reviewable in Git.
<code>make seed</code>	Creates the clearing account	It is infrastructure, not a customer, so it is created by script rather than an API call.
<code>ruff check .</code>	Lints	Catches style problems and likely bugs. Fast because it is written in Rust.
<code>pytest -q</code>	Runs the tests	<code>-q</code> is quiet output. Our 17 tests need no database, so they run in half a second.
<code>docker compose ps</code>	Lists container status	The <code>healthy</code> column is the one that matters – “running” is not the same as “ready”.

Diagnostic commands worth memorising

Command	Question it answers
<code>which <cmd></code>	Where would the shell find this?
<code>hash -r</code>	Clear the shell’s stale command cache
<code>echo \$PATH tr ':' '\n'</code>	Which folders does the shell search?
<code>ls -lt ~/Downloads head -5</code>	What did I most recently download?
<code>ls -la</code>	Show hidden files too
<code>python --version</code>	Which interpreter am I actually using?
<code>lsof -i :5432</code>	What is occupying this port?
<code>docker compose logs -f kafka</code>	Show me a container’s live output
<code>kill %1 %2 %3</code>	Stop jobs I backgrounded with <code>&</code>

`&` and `kill`

```
make relay & make worker & make api &
```

`&` sends a command to the background so the shell returns immediately. Three programs, one terminal.

They are still attached to that shell, and closing the terminal kills them. `kill %1 %2 %3` stops them by job number. Fine for a demo; in production each would be a separate managed service.

Part 7 - Where you actually are

Done and pushed: github.com/Aru-150905/payments-platform

- Compose stack: Postgres, Redis, Kafka in KRaft mode
- Double-entry ledger with balance and funds invariants
- Payment state machine with explicit legal transitions
- Idempotency enforced by database constraints
- Transactional outbox plus a relay process
- Kafka consumer with durable deduplication
- Alembic migrations, 17 unit tests, CI, four ADRs

Milestones 1 and 2 of 7.

Still to verify

Containers are up but these have not been run yet:

```
make topics && make migrate && make seed
ruff check . && pytest -q
```

Then the end-to-end proof:

```
make relay & make worker & make api &
sleep 4 && make demo
kill %1 %2 %3
```

In `make demo` output, look for: the payer balance going negative, the payee's rising by 250000, the repeated idempotency key returning the *same* payment, and the second capture returning `409`.

Next

Milestone 3 - retries with backoff, a dead-letter queue, and a replay tool. The prompt is ready in `docs/SESSION-PROMPTS.md`.

The question that follows immediately after anyone hears “Kafka” in an interview is “*what happens when a consumer cannot process a message?*” Right now the honest answer is “it gets logged and dropped”. M3 is the answer.

Glossary

Alembic – tool that applies versioned database schema changes.

Alpine – minimal Linux distribution; smaller, more secure container images.

Append-only – never edited or deleted, only added to. Corrections are new entries.

Argument – what you give a command to act on.

At-least-once – delivery guarantee where a message may arrive more than once but never zero times.

Atomic – happens completely or not at all; no partial state visible.

Bottle – Homebrew’s prebuilt binary. Fast, versus building from source.

Cask – a Homebrew package that is a normal Mac application.

CI (Continuous Integration) – automation that runs tests on every push.

Clearing account – holding account where money sits between authorization and capture.

Consumer group – set of consumers sharing an ID; they split partitions between them.

Container – isolated bundle of a program plus everything it needs.

Deadlock – two transactions each waiting for a lock the other holds. Neither can proceed.

Detached (-d) – run in the background, return the prompt.

DLQ (Dead Letter Queue) – where messages go after retries are exhausted.

Double-entry – recording every movement twice, summing to zero.

Effectively-once – at-least-once delivery plus idempotent consumers. The practical meaning of “exactly-once”.

Ephemeral – does not survive a restart.

Flag – a switch modifying a command, like `-p` or `-d`.

Formula – a Homebrew package that is a command-line tool.

Heap – memory a JVM program reserves for itself.

Idempotent – doing it twice has the same effect as doing it once.

Image – read-only container template. The class; a container is the object.

KRaft – Kafka’s built-in coordination mode, replacing Zookeeper.

Ledger – append-only record of money movements.

Linting – automated checking for style problems and likely bugs.

Listener – a network address Kafka accepts connections on and advertises.

Lock – claim on a database row so others must wait.

Migration – versioned, ordered schema change.

Minor units – smallest currency unit. Paise for INR.

Offset – a consumer’s position in a partition.

Outbox – table where events are written in the same transaction as the business change, then published separately.

Package manager – installs programs from a central catalogue.

Partition – a slice of a topic. Ordering is guaranteed within one only.

PATH – list of folders the shell searches for commands.

Pipe (|) – feeds one command's output into the next as input.

Port – numbered door on a machine.

Profile (Compose) – marks a service as opt-in rather than starting by default.

Relay – process that reads outbox rows and publishes them to Kafka.

Shell – program that reads and executes what you type. Yours is `zsh`.

State machine – explicit set of states and legal transitions between them.

Tag – version label on an image, after the colon.

Terminal – the window. Only the window.

Topic – a named Kafka stream.

Transaction (database) – group of changes that all succeed or all fail.

Venv – private folder with its own Python and packages.

Volume – Docker-managed storage that outlives the container.

Wheel – precompiled Python package, specific to a Python version and platform.

YAML – text format where indentation defines structure.